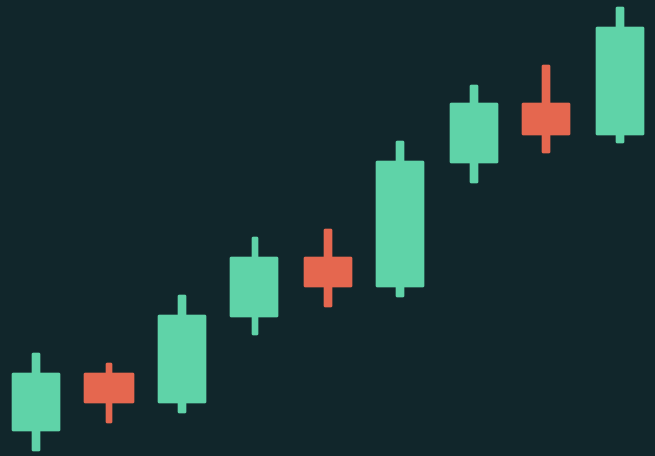
Stock Price Forecasting

StockSense AI: RNN and LSTM forecasting dashboard built with Python and Streamlit



Presented by Bhumika Saini and Vaishnavi

Educational project, not financial advice

Why we built this

Stock prices are noisy, and many beginner projects stop at a chart and one accuracy number.

StockSense AI goes further. It trains RNN and LSTM models on past closing prices, forecasts the coming sessions, and checks the result against a simple "tomorrow equals today" baseline.

The goal is an honest dashboard you can explore, not a trading tip.

2 models: Simple RNN and LSTM

60 past sessions fed in per prediction

1 naive baseline every model must beat

Project objectives

Explore market data

Pull NSE and US stock history and view it as close-price and candlestick charts.

Read the trend

Overlay SMA, EMA, RSI and MACD to see momentum before any modelling.

Forecast with deep learning

Train a Simple RNN or LSTM and predict the next session and several days ahead.

Judge it honestly

Score with MAE, RMSE and directional accuracy, always against a naive baseline.

Tools and technologies



Python

Core language for the whole pipeline



Pandas

Cleans data and computes indicators



Streamlit

Interactive dashboard in the browser



Plotly

Zoomable line and candlestick charts



TensorFlow

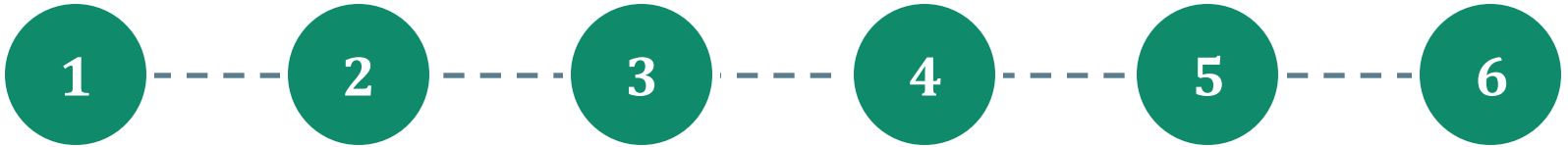
Builds and trains the RNN and LSTM



yfinance

Fetches NSE and US price history

How the app works



1	2	3	4	5	6
Fetch data	**Add indicators**	**Scale and split**	**Train**	**Evaluate**	**Forecast**
yfinance or offline demo set	SMA, EMA, RSI, MACD	70 / 15 / 15 in time order	RNN or LSTM, early stopping	MAE, RMSE, direction	Next session and multi-day

All of this runs from one sidebar and a single "Train & forecast" button.

Data and technical indicators

Where the data comes from

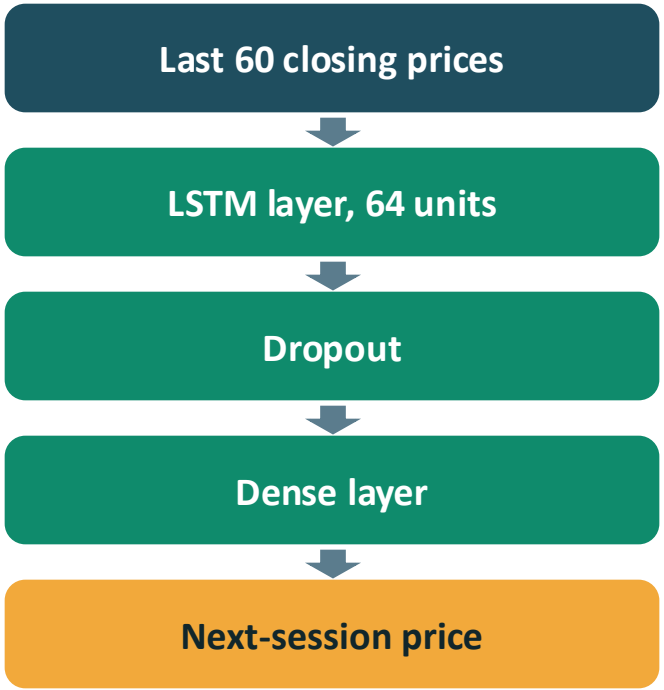
RELIANCE.NS TCS.NS INFY.NS

^NSEI AAPL MSFT NVDA

No internet? A clearly labelled synthetic demo dataset lets you try the app offline. It is not real market history.

Indicator	What it shows
SMA 20 / SMA 50	Short and medium-term trend
EMA 20	Trend, weighted to recent prices
RSI 14	Momentum: overbought or oversold
MACD + signal	Trend changes from moving-average gap

Model design: RNN vs LSTM



Simple RNN

Passes a hidden state along the sequence. Quick, but forgets long-range patterns.

LSTM (default)

Gates decide what to keep or forget, so useful signals last across the window.

Training safeguards

Early stopping and learning-rate reduction help stop overfitting.

Training setup and evaluation

Chronological split, so the model never sees the future



The scaler is fitted on the training set only, so nothing from validation or test leaks in.

MAE

Average error in price units

RMSE

Like MAE, but punishes big misses harder

Directional accuracy

How often up or down moves are called right

Naive baseline

Tomorrow = today's close. The model must beat it

Dashboard features

- Interactive close-price and candlestick charts
- Next-session and multi-day forecasts
- Model metrics shown next to the baseline
- CSV downloads: history, forecasts, test predictions
- Sidebar to pick ticker, period and model
- Starter tests written with pytest

Project structure

```
app.py
requirements.txt
run_windows.bat
src/
  data_loader.py
  indicators.py
  preprocessing.py
  model_builder.py
  train.py
  evaluate.py
  visualization.py
tests/
```

Limitations and what's next

Limitations

- Prices alone miss news and macro events
- Recursive multi-day forecasts pile up error
- Forecast dates ignore exchange holidays

Next steps

- Walk-forward validation
- Multi-feature models using OHLCV
- Prediction intervals and more baselines

For education and experimentation only, not financial advice. Thank you!